

Manual Técnico
Módulo Aluminio

Versión 1.0 | 22/05/2026

Documentación técnica para desarrolladores y administradores del sistema.

Índice de Contenido

#	Sección
1.	Arquitectura del Sistema
2.	Estructura de Archivos
3.	Base de Datos
3.1	Esquema de la Tabla
3.2	Campos de Precio
3.3	Relaciones
4.	Dependencias
5.	Módulos y Clases Principales
5.1	DBManager
5.2	PantallaMaterialesAluminio
5.3	Parsers
6.	Flujo de Datos
6.1	Guardado Manual
6.2	Importación
6.3	Edición
7.	Manejo de Errores
8.	Logging y Depuración
9.	Configuración
10.	Formatos de Importación

1. Arquitectura del Sistema

El Módulo de Aluminio sigue una arquitectura de tres capas:

Capa	Tecnología	Responsabilidad
Presentación	PyQt6	Interfaz gráfica (PantallaMaterialesAluminio)
Lógica de Negocio	Python 3.12	DBManager, validaciones, reglas de importación
Acceso a Datos	pyodbc + SQL Server	CRUD sobre materiales_aluminio
Almacenamiento	SQL Server	Base de datos 'aluminio', tabla materiales_aluminio

[Imagen pendiente: arquitectura_sistema.png]

2. Estructura de Archivos

Archivo	Descripción
pantalla_materiales_aluminio.py	UI principal, modos de operación, eventos
db_manager.py	CRUD, conexión BD, merge_max, listar_todos
db.py	OBSOLETO - No utilizado (mantener por compatibilidad)
imprimir.py	Generación de PDF con reportlab
utils.py	Funciones: dec2, merge_max, ordenar_registros_asc, CAMPOS_PRECIO_BD
restricciones.py	Reglas de negocio: is_title_like, desc_bloqueada, registro_todo_ceros
parsers/parse_excel.py	Importación desde Excel (formato Profilo)
parsers/parse_pdf.py	Importación desde PDF (formato Mateco)
parsers/parse_csv.py	Importación desde CSV genérico

[Imagen pendiente: estructura_modulos.png]

3. Base de Datos

3.1 Esquema de la Tabla

Columna	Tipo	Nulo	Descripción
clave	varchar(10)	NO	PK - Clave única del perfil
descripcion	varchar(100)	NO	Nombre comercial
largo	decimal(4,2)	NO	Largo en metros (6.10, 5.50, etc.)
sa	decimal(10,2)	YES	Precio S/A (Sin acabado)
nat	decimal(10,2)	YES	Precio NAT (Natural)
bmo	decimal(10,2)	YES	Precio BMO (Bronce mate)
e100	decimal(10,2)	YES	Precio E-100
e200	decimal(10,2)	YES	Precio E-200
e400	decimal(10,2)	YES	Precio E-400
bco	decimal(10,2)	YES	Precio BCO (Blanco)
hue	decimal(10,2)	YES	Precio HUESO
np	decimal(10,2)	YES	Precio NP (Negro punta)
gris	decimal(10,2)	YES	Precio GRIS
cho	decimal(10,2)	YES	Precio CHO (Chocolate)
ace	decimal(10,2)	YES	Precio ACE (Acero)
madl	decimal(10,2)	YES	Precio MAD L (Madera línea)
madt	decimal(10,2)	YES	Precio MAD T (Madera textura)

[Imagen pendiente: diagrama_bd.png]

3.2 Campos de Precio

```
# Constante en utils.py
CAMPOS_PRECIO_BD = [
    "sa", "nat", "bmo",
    "e100", "e200", "e400",
    "bco", "hue", "np",
    "gris", "cho", "ace",
    "madl", "madt"
]
```

3.3 Relaciones

- La tabla NO tiene claves foráneas explícitas.
- La clave primaria es 'clave' (única).
- No hay relaciones con otras tablas (independiente).
- La integridad se maneja a nivel de aplicación.

4. Dependencias

Librería	Versión	Uso
PyQt6	6.5+	Interfaz gráfica completa
pyodbc	4.0+	Conexión ODBC a SQL Server
openpyxl	3.0+	Lectura de archivos Excel (Profilo)
pdfplumber	0.9+	Extracción de texto desde PDF (Mateco)
reportlab	3.6+	Generación de PDF para impresión
win32print	pywin32	Impresión directa a impresora física
Python	3.10+	Lenguaje base

```
pip install PyQt6 pyodbc openpyxl pdfplumber reportlab pywin32
```

5. Módulos y Clases Principales

5.1 DBManager (db_manager.py)

- listar_todos() → List[Dict] - Obtiene todos los materiales
- obtener_por_clave(clave) → Dict - Busca por clave
- existe_clave(clave) → bool - Verifica existencia
- insertar_o_actualizar(material, use_merge=False) → Dict - Guarda material
- eliminar_por_clave(clave) → None - Elimina material

```
# Ejemplo de uso
db = DBManager(db_name="aluminio")
material = {
    "clave": "7821C",
    "descripcion": "PERFIL VENTANA",
    "largo": Decimal("6.10"),
    "sa": Decimal("125.50"),
    ...
}
db.insertar_o_actualizar(material, use_merge=False)
```

5.2 PantallaMaterialesAluminio (pantalla_materiales_aluminio.py)

- _on_guardar() - Guarda material (modo nuevo o edición)
- _modo_editar() - Activa modo edición, habilita búsqueda
- _modo_eliminar() - Activa modo eliminación
- Inicio() - Limpia todos los campos y tabla
- _buscar_dinamico() - Filtra tabla mientras escribe
- _on_imprimir() - Diálogo de impresión (Pantalla/PDF/Impresora)
- _on_importar() - Importa desde archivo externo
- eventFilter() - Valida clave duplicada al perder foco
- _llenar_tabla() - Carga datos en QTableWidget

```
# Atajos de teclado definidos
Ctrl+S → _on_guardar
Ctrl+F → focus búsqueda
Ctrl+P → _on_imprimir
Delete → _modo_eliminar
F5 → _on_refresh
Esc → _on_salir
```

5.3 Parsers (parsers/)

parse_excel.py

- parse_excel_profilo(filename) → List[Dict] - Extrae materiales de Excel
- Mapea encabezados (SA → sa, NAT → nat, ELE → e100/e200/e400)
- Aplica reglas de negocio (ignora títulos, claves C1150/C1200)
- Aplica EXCEPCIONES_LARGO para claves específicas

parse_pdf.py

- parse_pdf_mateco(pdf_path) → List[Dict] - Extrae materiales de PDF Mateco
- Reconoce largos válidos: 6.10, 5.50, 5.00, 4.60, 3.66
- Asigna precios en orden: SA, NAT, BMO, ELE, BCO, HUESO, NP, GRIS, MADL, MADT
- ELE se distribuye a e100, e200, e400

parse_csv.py

- parse_csv_generico(filename) → List[Dict] - Importa desde CSV
- Espera encabezados: CLAVE, DESCRIPCION, SA, NAT, ...
- Aplica mismas restricciones que Excel/PDF

6. Flujo de Datos

6.1 Guardado Manual

- 1. Usuario completa campos (Clave, Descripción, Largo, Precios)
- 2. Usuario presiona Guardar (Ctrl+S)
- 3. UI valida campos obligatorios (Clave no vacía, Descripción no vacía)
- 4. UI llama a DBManager.insertar_o_actualizar(use_merge=False)
- 5. DBManager verifica si la clave existe con SELECT
- 6. Si existe → UPDATE, si no → INSERT
- 7. UI limpia pantalla (Inicio())

[Imagen pendiente: flujo_datos.png]

6.2 Importación desde Archivo

- 1. Usuario selecciona Importar y elige archivo (.xlsx, .pdf, .csv)
- 2. UI llama al parser correspondiente (parse_excel_profilo, etc.)
- 3. Parser extrae registros aplicando reglas de negocio
- 4. Para cada registro, DBManager.insertar_o_actualizar(use_merge=True)
- 5. DBManager aplica merge_max (conserva precio más alto entre existente y nuevo)
- 6. UI muestra mensaje con número de registros importados

6.3 Edición

- 1. Usuario presiona Editar
- 2. UI activa barra de búsqueda y limpia tabla
- 3. Usuario escribe texto en búsqueda
- 4. UI filtra y muestra resultados
- 5. Usuario hace doble clic en registro deseado
- 6. UI carga datos (Clave, Descripción deshabilitadas, Precios habilitados)
- 7. Usuario modifica precios o largo
- 8. Usuario presiona Guardar → UPDATE en BD
- 9. UI limpia pantalla

7. Manejo de Errores

Nivel	Mecanismo	Ejemplo
Validación UI	Comprobaciones en _on_guardar()	Clave vacía → mensaje
Clave duplicada	eventFilter() + QMessageBox.question	Pregunta si editar
Conexión BD	try/except en DBManager	Logging + raise
Importación	try/except por registro	Logging + continúa
Impresión	try/except en imprimir_pdf_guardar	QMessageBox.critical

```
# Ejemplo de manejo de errores en db_manager.py
with pyodbc.connect(self.conn_str) as cn:
    try:
        cur = cn.cursor()
        cur.execute(sql, vals)
        cn.commit()
    except pyodbc.Error as e:
        logger.error(f"Error SQL: {e}")
        raise
```

8. Logging y Depuración

- Archivo de log: D:\vah\materiales\aluminio\aluminio_db.log
- Tamaño máximo: 1 MB
- Backup: 3 archivos rotativos
- Encoding: UTF-8
- Nivel: DEBUG (registra errores de conexión y operaciones)

```
# Configuración en db.py (obsoleto) y db_manager.py
logger = logging.getLogger("aluminio_db")
logger.setLevel(logging.DEBUG)
handler = RotatingFileHandler(
    "aluminio_db.log", maxBytes=1_000_000, backupCount=3, encoding="utf-8"
)
logger.addHandler(handler)
```

■ ***Nota:*** *El log principal del módulo se encuentra en aluminio_db.log. Revise este archivo ante errores de conexión o importación.*

9. Configuración

Cadena de conexión a SQL Server (db_manager.py):

```
self.conn_str = (
    "Driver={ODBC Driver 18 for SQL Server};"
    "Server=DESKTOP-AE60PIU\\SQLEXPRESS;"
    f"Database={db_name};"
    "Trusted_Connection=yes;"
    "Encrypt=no;"
)
```

Configuración de largos disponibles:

```
# pantalla_materiales_aluminio.py
self.largo_input.addItems(["6.10 MTS.", "5.50 MTS.", "5.00 MTS.", "4.60 MTS.", "3.66 MTS.
"])
```

Reglas de excepción de largo (parse_excel.py):

```
EXCEPCIONES_LARGO = {
    "7821C": 4.60,
    "9955C": 4.60,
    ...
}
```

10. Formatos de Importación

10.1 Excel (Profilo)

- Formato esperado: fila 11 = encabezados
- Encabezados: CLAVE, DESCRIPCION, SA, NAT, BMO, E1, E2, E4, BL, HU, NP, GR, CH, AC, NO, MA
- Datos desde fila 12
- Largo por defecto: 6.10 m, excepto claves en EXCEPCIONES_LARGO
- E1/E2/E4 → e100/e200/e400

10.2 PDF (Mateco)

- Formato esperado: líneas con clave, descripción, largo y precios
- Largos válidos: 6.10, 5.50, 5.00, 4.60, 3.66
- Orden de precios: SA, NAT, BMO, ELE, BCO, HUESO, NP, GRIS, MADL, MADT
- ELE se distribuye a e100/e200/e400
- Ignora líneas que comienzan con * o contienen meses (ENE, FEB, ...)

10.3 CSV

- Formato esperado: encabezados en primera fila
- Columnas: CLAVE, DESCRIPCION, SA, NAT, BMO, E100, E200, E400, BCO, HUESO, NP, GRIS, CHO, ACE, MADL, MADT
- Delimitador: coma (,)
- Encoding: UTF-8

— Fin del Manual Técnico —

Documento generado el 22/05/2026 | Versión 1.0

VAH ERP © Todos los derechos reservados